

Lectura 3

Introducción a MATLAB/Octave/Scilab

Segunda Parte. Funciones.

Dr. Luciano Ponzellini Marinelli
lucianoponzellini@uca.edu.ar

Métodos y Cómputos Numéricos
Licenciatura en Ciencias de Datos
Facultad de Química e Ingeniería
Rosario, Argentina

Notas de clase
12 agosto 2025





Organización

FUNCIONES

SCRIPTS

PLOTEOS

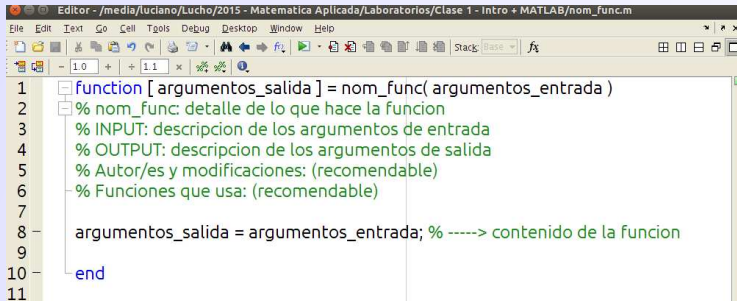
FUNCIONES

Funciones definidas por usuarios

Hay dos modos básicos de definir funciones en MATLAB/Ocative/Scilab:

- en archivos `.m` (*m-file functions*) o `.sci` (*Scilab*).
- en líneas de comandos (*inline*, *function_handle*, *deff*).

Funciones definidas en archivos: m-file functions



```

Editor - /media/luciano/Lucho/2015 - Matematica Aplicada/Laboratorios/Clase 1 - Intro + MATLAB/nom_func.m
File Edit Text Go Cell Tools Debug Desktop Window Help
[Icons] Stack: Base fx
- 1.0 + ÷ 1.1 × [Icons]
1 function [ argumentos_salida ] = nom_func( argumentos_entrada )
2 % nom_func: detalle de lo que hace la funcion
3 % INPUT: descripcion de los argumentos de entrada
4 % OUTPUT: descripcion de los argumentos de salida
5 % Autor/es y modificaciones: (recomendable)
6 % Funciones que usa: (recomendable)
7
8 argumentos_salida = argumentos_entrada; % -----> contenido de la funcion
9
10 end
11

```

Funciones definidas en archivos

La primera línea es su definición:

```
function [argumento(s) salida] = nombre(argumento(s) entrada)
```

- **nombre:** *Es el nombre de la función.*
Tiene que coincidir con el nombre del archivo nombre.m.
No debe incluir espacios ni palabras reservadas por el software.
- **argumento(s) entrada:** *Parámetro(s) que le paso a la función.*
Si hay más de uno, van entre paréntesis separados por comas.
Si no hay, no hace falta poner paréntesis.
- **argumento(s) salida:** *Parámetro(s) devuelto por la función.*
Si hay más de uno van entre corchetes separados por comas.
Si sólo hay un valor no hacen falta corchetes.
Si no hay se omiten corchetes y el signo igual (=).

Funciones definidas en archivos

- Cuando la función no devuelve ningún valor:

```
function nombre(argumentos_entrada)
```

- Cuando la función no precisa variables de entrada:

```
function [argumentos_salida] = nombre
```

- Cuando la función devuelve un solo resultado:

```
function [y] = nombre(a,b,c)
```

```
function y = nombre(a,b,c)
```

- En general, definiremos la función del siguiente modo:

```
function [y1,y2] = nombre_funcion(a,b,c)
```

Funciones definidas en archivos

- Una función no modifica los argumentos que recibe: **strings**, **escalares**, **vectores**, **matrices** o incluso otras funciones.
- Opcionalmente (pero recomendable) desde la segunda línea se ponen **comentarios** que detallen la función.
- **Help de mi función:** Cuando se teclea en la Ventana de Comandos:

```
>> help nom_func
```

se muestran los comentarios de las primeras líneas del archivo **nom_func.m** que comienzan con %.

- Los valores de los **argumentos de salida** deben calcularse dentro de la función.
- Las variables definidas dentro de la función son **variables locales** que pertenecen a dicho espacio de trabajo (workspace) y no son vistas desde otros espacios de trabajo.
- Opcionalmente (pero recomendable) terminar la función con **end**.

Funciones en línea: *inline functions* (MATLAB/Octave)

Funciones en línea: Son funciones de una línea de comandos definidas por una cadena de caracteres (string).

```
arg_func = inline('expresión','arg_1','arg_2',...)
```

- **arg_func:** *Es el argumento de salida de la función.*
Sólo admite un argumento de salida.
- **expresión:** *Expresión de la función.*
Se pasa como cadena string, por eso va entre comillas.
- **'arg_1','arg_2',...:** *Argumento(s) de entrada de la función.*
Es el o los argumentos que aparecen como variable en la expresión.
Se pasan como string entre comillas separados por coma.
Si hay un solo argumento no hace falta especificarlo.
- Las funciones inline no admiten parámetros en su expresión.

Funciones en línea: *deff functions* (Scilab)

Funciones en línea: Son funciones de una línea de comandos definidas por una cadena de caracteres (string).

```
deff('arg_func = nom_func(arg_1,arg_2,...)','expresión')
```

- **arg_func:** *Es el argumento de salida de la función.*
Sólo admite un argumento de salida.
- **nom_func:** *Es el nombre de la función (sin espacios).*
- **'arg_1','arg_2',...:** *Argumento(s) de entrada de la función.*
Es el o los argumentos que aparecen como variable en la expresión.
- **expresión:** *Expresión de la función.*
Se pasa como cadena string, por eso va entre comillas.
- Las funciones deff no admiten parámetros en su expresión.

Ejemplos de funciones en línea

MATLAB/Octave:

```
>> f=inline('x^2+3')
f = Inline function:
f(x) = x^2+3      (defino la función cuadrática  $f(x) = x^2 + 3$ )
>> f(3)
ans = 12
>> g=inline('x^3+y^3','x','y')
g = Inline function:
g(x,y) = x^3+y^3  (defino el campo  $g(x,y) = x^3 + y^3$ )
>> g(2,5)
ans = 133
```

Scilab:

```
deff('y=func(x)','y=x^2+2')  (defino la cuadrática  $f(x) = x^2 + 2$ )
func(3.5)
ans=14.25
```

Funciones anónimas: *function handles* (MATLAB/Octave)

Funciones anónimas: Fueron introducidas desde MATLAB 7.0
Estos objetos se llaman anónimos porque su construcción no usa strings
Están asociados a una variable cuyo tipo es *function_handle* con el **@**:

```
nom_func = @(arg_1,arg_2,...) expresión
```

- **nom_func:** *Es el valor de salida de la función.*
Sólo admite un argumento de salida.
- **arg_1,arg_2,...:** *Argumento(s) de entrada de la función.*
Es el o los argumentos que aparecen como variable en la expresión.
Se pasan separados por comas.
- **expresión:** *Expresión de la función.*
Solo admite una expresión ejecutable.
- Las funciones anónimas sí admiten parámetros en su expresión.

Funciones anónimas: *function handles* (MATLAB/Octave)

```
>> f=@(x) x^2+3  
f = @(x) x^2+3      (defino la función cuadrática  $f(x) = x^2 + 3$ )  
  
>> f(3)  
ans = 12  
  
>> g=@(x,y) x^3 + y^3  
g = @(x,y) x^3 + y^3      (defino el campo  $g(x, y) = x^3 + y^3$ )  
  
>> g(2,5)  
ans = 133
```

Algunos help de versiones nuevas de MATLAB advierten que las funciones 'inline' van a ser removidas en versiones futuras.

Pasando funciones como argumentos en MATLAB

- Es muy común cuando programamos querer pasar como argumento de entrada de una función otra función.
- Según Moler en *“Numerical Computing with MATLAB”* [Mole2015], hay al menos cinco formas diferentes de pasar una función como argumento de otra función en MATLAB 7.0 o versiones posteriores:
 - ① Funciones en línea (inline).
 - ② Definidas en un archivo-m (m-file).
 - ③ Funciones anónimas (function handle).
 - ④ Expresiones de cadena de caracteres.
 - ⑤ Expresiones simbólicas (syms).

Veremos más adelante como pasar funciones como argumentos.

SCRIPTS

Trabajar con scripts (guiones o libretos)

- Los **scripts (archivos de comandos)** son archivos con una secuencia de órdenes o comandos semejante a la que se ingresan por teclado.
- No tienen argumentos de entrada ni de salida como las funciones.
- Los comandos se ejecutan *secuencialmente* cuando se teclea el nombre del script (sin extensión) en la Ventana de Comandos.
- El script debe almacenarse en el directorio actual donde trabajamos.
- Conviene poner punto y coma (;) al final de cada sentencia para evitar una salida innecesaria por la Ventada de Comandos.
- Un archivo (script o función) puede llamar otros archivos.
- En MATLAB/Octave son de extensión .m y en Scilab .sce.

PLOTEOS

¿Cómo graficar en 2-D?

MATLAB/Octave tienen cuatro funciones básicas para crear gráficos 2D:

- **plot** crea un gráfico a partir de vectores y/o columnas de matrices con escalas lineales sobre ambos ejes (idem en Scilab).
- **semilogx** Idem con escala lineal en el eje de ordenadas y logarítmica en el eje de abscisas.
- **semilogy** Idem con escala lineal en el eje de abscisas y logarítmica en el eje de ordenadas.
- **loglog** Idem con escala logarítmica en ambos ejes.

*En lo sucesivo usaremos la función **plot**
pero puede consultarse el Help las restantes en modo similar*

Ploteos 2D en MATLAB/Octave: el comando *plot*

```
>> x = [1 6 5 2 1];
>> y = [1 0 4 3 1];
>> plot(x,y), grid on,
>>
>> z = [1 3 2 4 5 3];
>> plot(z), grid on,
```

(se genera una Figura)
(grid on activa una grilla en la gráfica)
(por defecto x=[1 2 3 4 5 6])
(se genera otra Figura)

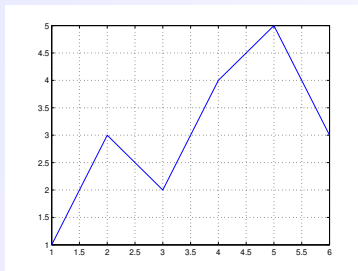
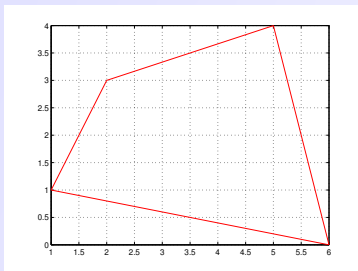


Figura: Ploteos sencillos: poligonal cerrada y poligonal abierta.

Ploteos 2D en MATLAB/Octave: el comando *plot*

```
>> x = -2:.01:2; (defino una discretización en [-2,2])
>> plot(x,exp(x),'g'), grid on
>> x=-2*pi:pi/100:2*pi; (defino una discretización en [-2π,2π])
>> y = sin(x); z = cos(x);
>> plot(x,y,'c',x,z,'m'), grid on, (ambas curvas en la misma Figura)
```

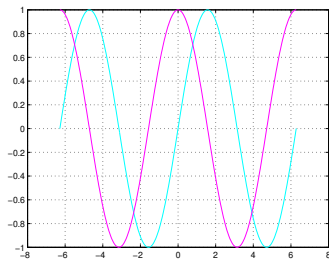
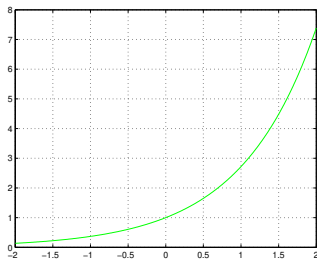


Figura: Ploteos de las funciones e^x , $\text{sen}(x)$ y $\text{cos}(x)$.

Ploteos 2D: *colores, estilos y marcadores*

- Algunos colores, estilos y marcadores usuales del *plot* son:

Color	Símbolo
Azul	b
Verde	g
Rojo	r
Cyan	c
Magenta	m
Amarillo	y
Negro	k

Tipo de línea	Símbolo
Continua	-
A trazos	-
Punteada	:
A trazos y puntos	-.

Tipo de marca	Símbolo
Puntos	.
Círculos	o
Cruces	x
Sumas	+
Asteriscos	*

Ploteos 2D: colores, estilos y marcadores

```
>> x = 0:0.4:10;                                (defino una discretización en [0, 10])
>> y1 = sin(x).*exp(-0.4*x);                     (defino y1 como vector)
>> y2 = cos(x).*exp(-0.5*x);                     (defino y2 como vector)
>> plot(x,y1,'b+',x,y2,'ro'), grid on
```

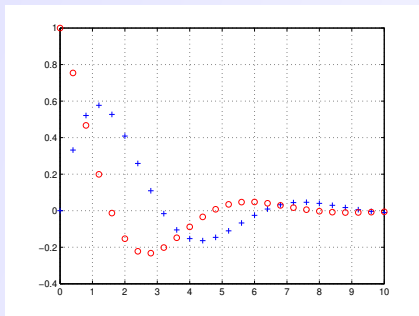


Figura: Distintos estilos de $y_1(x) = \sin(x)e^{-0,4x}$ y $y_2(x) = \cos(x)e^{-0,5x}$.

Ploteos 2D en MATLAB/Octave: la función *subplot*

Una ventana gráfica se puede dividir en **m** particiones horizontales y **n** verticales. Para esto usamos la función **subplot(m,n,i)** donde:

- **m** número de subdivisiones de filas.
- **n** número de subdivisiones de columnas.
- **i** subdivisión activa donde grafico.

```
>> x = 0:.5:10;
>> y = sin(x);
>> z = cos(x);
>> v = exp(-x*.1).*y;
>> w = y.*z;
>> subplot(2,2,1), plot(x,y)
>> subplot(2,2,2), plot(x,z,'r')
>> subplot(2,2,3), plot(x,v,'c')
>> subplot(2,2,4), plot(x,w,'g')
```

(defino una discretización en [0, 10])

(defino y, z, v, w como vectores)

(subplots en una misma Figura)

Ploteos 2D en MATLAB/Octave: la función *subplot*

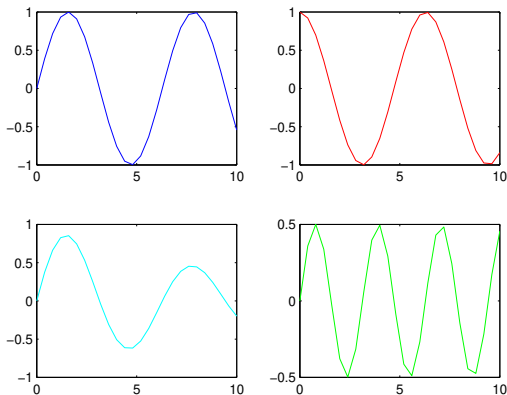


Figura: Subplot 2x2 de $y = \sin(x)$ (azul), $z = \cos(x)$ (red), $v = e^{-0,1x} \sin(x)$ (cian) y $w = \sin(x)\cos(x)$ (verde).

Ploteos 2D en MATLAB/Octave: *contorno de funciones*

La función **contour**(**X,Y,Z,nivel**) sirve para graficar *curvas de nivel* de un campo escalar $z = f(x, y)$:

- **X,Y** coordenadas x e y en arreglos bidimensionales.
- **Z** arreglo bidimensional con valores de la función.
- **nivel** vector que contiene los niveles de contorno ó entero que representa el número de niveles de contorno (se divide el valor mínimo y máximo de Z en $m-1$ niveles).

```
>> x = -2:0.2:2;
>> y = x;
>> [X,Y] = meshgrid(x,y); (uso meshgrid para hacer el "mallado")
>> Z = X.*exp(-X.^2-Y.^2); (defino Z como matriz)
>> contour(X,Y,Z,10), grid on (hago 10 curvas de nivel)
```

Ploteos 2D en MATLAB/Octave: *contorno de funciones*

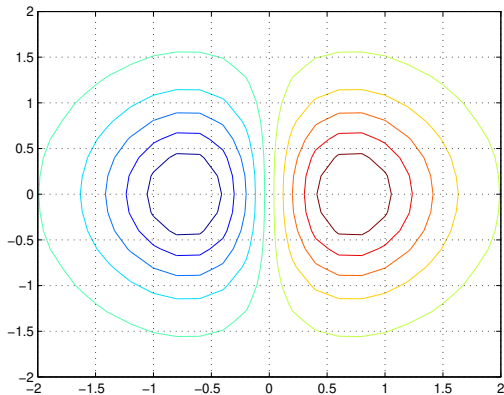


Figura: Curvas de nivel de $z = xe^{-(x^2+y^2)}$.

Ploteos 2D en MATLAB/Octave: *Contorno de funciones*

- También podemos utilizarlo para graficar funciones implícitas $f(x, y) = 0$ en 2D, por ejemplo, una parábola y una elipse:

$$\begin{cases} x^2 - 2x - y + \frac{1}{2} = 0 \\ x^2 + 4y^2 = 4 \end{cases}$$

```
>> x = -3:0.1:3;
>> y = x;
>> [X,Y] = meshgrid(x,y);
>> Z1 = X.^2-2*X-Y+0.5;           (defino Z1 como matriz)
>> Z2 = X.^2+4*Y.^2-4;           (defino Z2 como matriz)
>> contour(X,Y,Z1,[0,0],'b'), hold on,   (hold on mantiene la Figura)
>> contour(X,Y,Z2,[0,0],'r'), grid on,
```

*El vector [0,0] como argumento de contour especifica el nivel de contorno.
Los niveles de contorno se ingresan en forma de vector.
Solo nos interesa el contorno de nivel $z = 0$*

Contorno de funciones

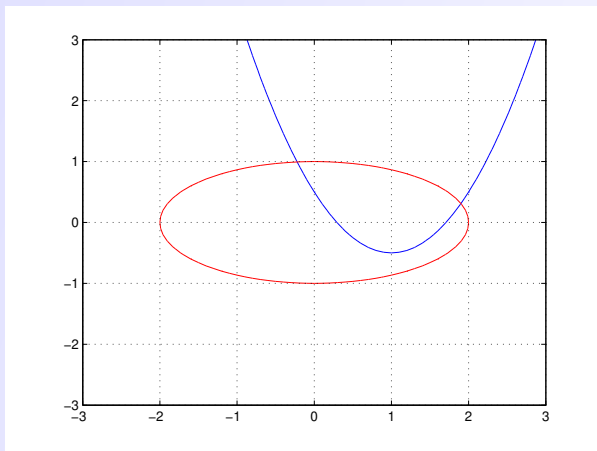


Figura: Intersección de la parábola $x^2 - 2x - y + \frac{1}{2} = 0$ y la elipse $x^2 + 4y^2 = 4$.

Ploteos 3D en MATLAB/Octave

Pasos para graficar el campo en dos variables:

$$z = f(x, y)$$

sobre un dominio rectangular $[a, b] \times [c, d]$.

- ① Definimos **x**, **y** vectores con coordenadas en una y otra dirección de la retícula (grilla o malla) dominio del campo.
- ② Creamos dos matrices **X** (con filas copias de **x**) e **Y** (con columnas copias de **y**) con **meshgrid** y representan las coordenadas de **x** e **y** de todos los puntos de la retícula.
- ③ La matriz de valores **Z** del campo z se calcula a partir de las matrices de coordenadas **X** e **Y**.
- ④ Ejecutamos la función **mesh** con las matrices **X**, **Y** y **Z** como argumentos. Se plotea una superficie cuyos puntos con coordenadas

$$(X(i,j), Y(i,j), Z(i,j))$$

Ploteos 3D en MATLAB/Octave: la función *mesh*

```
>> x = -2:0.05:2;
>> y = -1:0.1:1;
>> [X,Y] = meshgrid(x,y);
>> Z = X.*exp(-X.^2-Y.^2);
>> mesh(X,Y,Z)
```

(abre una Figura y plotea la superficie)

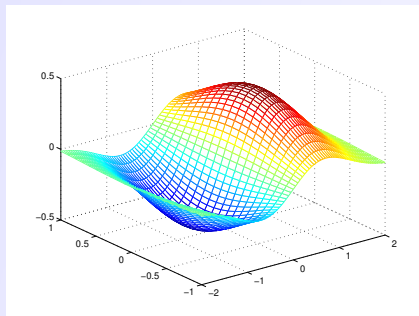


Figura: Mesh de la superficie de $z = x e^{-(x^2+y^2)}$.

Ploteos 3D en MATLAB/Octave: la función *surf*

- **mesh** elimina las líneas ocultas (el color depende del valor z), en cambio usando **surf**, aparece una superficie faceteada.
- El color de las facetas depende del valor de la función.

`>> surf(X,Y,Z), colorbar` (plotea la superficie faceteada en una Figura y **colorbar** agrega una barra de color con escalas)

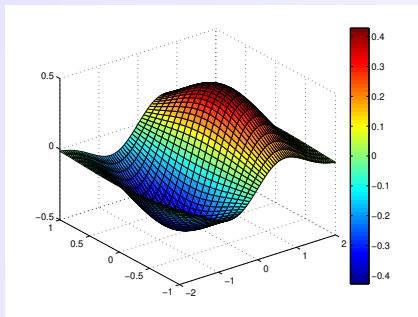


Figura: Surf de la superficie de $z = x e^{-(x^2+y^2)}$.

Bibliografía

- **[Ette1998]** D.M. Etter, *Solución de problemas de ingeniería con MATLAB*, Pearson Prentice Hall, Madrid, España, 1998.
- **[MaFi2000]** J.M. Mathews & K.D. Fink, *Métodos Numéricos con MATLAB*, 3era Ed., Pearson Prentice Hall, Madrid, 2000.
- **[Mole2015]** C.B. Moler, *Numerical computing with MATLAB*, SIAM, Philadelphia, 2015.
- **[Ponz2024]** L. Ponzellini Marinelli, *Taller de Álgebra Lineal en Scilab*, FCEIA, UNR, 2024.
- **[SPMK2023]** J.A. Semitiel, L. Ponzellini Marinelli & A.I. Kurdobrin, *Notas de Clases de Matemática Aplicada de Ingeniería Eléctrica e Ingeniería Electrónica*, FCEIA, UNR, 2023.
- **[SiSo2010]** J.W. Signorelli, J. Sorribas, *Notas de Clases de Informática Aplicada y Métodos Computacionales*, FCEIA, UNR, 2010.